

Computational Astrophysics A2 - Ayaan Dutt

September 27, 2025

```
[2]: import numpy as np
import matplotlib.pyplot as plt
```

1 Trajectory of a Volcanic Boulder

A boulder is ejected out of the summit of a volcano, travels for 10s, and then impacts the side of the mountain at a location 150m below the summit and 1280m horizontally from the point of ejection. Assuming that the boulder is a point mass (no rotational degrees of freedom) and it only experiences the force of gravity while travelling through the air, we can solve for its trajectory analytically using Newton's laws of motion:

$$s = ut + \frac{1}{2}at^2$$
$$v = u + at$$

Decomposing the position and velocity vectors along the x and y axis, we can determine the initial and final velocities. We are given $t = 10s$, $x = 1280m$, $y = -150m$, $a_x = 0m/s^2$ and $a_y = -g = -9.8m/s^2$ since there is only gravitational acceleration acting on the body.

$$v_x(0) = \frac{x}{t} \Rightarrow \boxed{v_x(0) = 128 \text{ m/s}}$$
$$v_y(0) = \left(\frac{y}{t} + \frac{gt}{2}\right) \Rightarrow \boxed{v_y(0) = 34 \text{ m/s}}$$

Using these in the second equation:

$$v_x(10) = v_x(0) \Rightarrow \boxed{v_x(10) = 128 \text{ m/s}}$$
$$v_y(10) = v_y(0) - gt \Rightarrow \boxed{v_y(10) = -64 \text{ m/s}}$$

Hence, the trajectory of the boulder is described by:

$$\boxed{x(t) = 128t}$$
$$\boxed{y(t) = 34t - 4.9t^2}$$

I will now try to reproduce these results using two numerical methods and compare their accuracy and efficiency - (1) Shooting Method and (2) Relaxation Method.

1.1 Shooting Method

Let us define an abstract state vector that includes both the position and velocity information along both axes, and write our differential equation in terms of it:

$$\vec{u} = \begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix} \Rightarrow \frac{d\vec{u}}{dt} = \begin{pmatrix} v_x \\ v_y \\ 0 \\ -g \end{pmatrix} = \vec{f}(\vec{u})$$

We are given the boundary conditions:

$$B_1 = \begin{pmatrix} x(0) - 0 \\ y(0) - 0 \end{pmatrix} = 0$$

$$B_2 = \begin{pmatrix} x(10) - 1280 \\ y(10) + 150 \end{pmatrix} = 0$$

However, we do not know the velocity at either end. Hence, we will insert random guess values for the initial velocity $\vec{V}_g = \begin{pmatrix} v_x^{(g)}(0) \\ v_y^{(g)}(0) \end{pmatrix}$ where the superscript g denotes a guess. This allows us to solve a boundary value problem with the following equation, using RK4 integration:

$$\frac{d\vec{u}}{dt} = \vec{f}(\vec{u})$$

With the initial condition $\vec{u}_0 = \begin{pmatrix} 0 \\ 0 \\ v_x^{(g)}(0) \\ v_y^{(g)}(0) \end{pmatrix}$. The solution gives us the final state $\vec{u} = \begin{pmatrix} x_f \\ y_f \\ v_{xf} \\ v_{yf} \end{pmatrix}$ which

we can use to define a discrepancy vector in terms of the second boundary condition:

$$\vec{F} = \begin{pmatrix} x_f - 1280 \\ y_f + 150 \end{pmatrix}$$

Note that when our guess values match the correct velocity values, this vector should go to zero. In order to determine the correction factor, we assume a correction vector $\delta\vec{V}_g$ such that $\vec{F}(\vec{V}_g + \delta\vec{V}_g) \approx 0$. This is determined from Newton's method:

$$[J] \cdot \delta\vec{V}_g = -\vec{F}(\vec{V}_g) \text{ where } J_{ij} = \frac{\partial F_i}{\partial V_j} = \frac{F_i(V_j + \Delta V_j) - F_i(V_j)}{\Delta V_j}$$

Where F_i are the components of $\vec{F}$ and V_j are the components of $\vec{V}_g$. After determining this Jacobian, we can invert it to compute $\delta\vec{V}_g$ and put it back into our initial guess to get a better final vector u_f .

We can check the accuracy of our method by measuring the new discrepancy vector $\vec{F}(\vec{V}_g + \delta\vec{V}_g)$ and iterate until it is within a desired tolerance ϵ .

```
[3]: '''
      Iterative shooting method
      '''

      # Parameters
      g = 9.8
      T = 10
      N = 1000
      dt = T/N

      # Boundary conditions
      x0, y0 = 0, 0
      xf, yf = 1280, -150
      vx_guess = 500
      vy_guess = -200
      u0 = np.array([x0, y0, vx_guess, vy_guess])

      # Derivate function
      def f(u):
          x, y, vx, vy = u
          return np.array([vx, vy, 0, -g])

      # RK4 integrator
      def rk4(u0):
          u = u0.copy()
          for i in range(N):
              k1 = f(u)
              k2 = f(u + 0.5*dt*k1)
              k3 = f(u + 0.5*dt*k2)
              k4 = f(u + dt*k3)
              u = u + (dt/6)*(k1 + 2*k2 + 2*k3 + k4)
          return u

      # Final state for a guess velocity
      def final_state(vx, vy):
          u0 = np.array([0.0, 0.0, vx, vy])
          u_final = rk4(u0)
          return u_final

      # Discrepancy vector
      def F(vx, vy):
          u_final = final_state(vx, vy)
          x_final, y_final = u_final[0], u_final[1]
          return np.array([x_final - xf, y_final - yf])

      # Jacobian of F wrt V_guess
      def shooting_jacobian(vx, vy, h=1e-3):
          F0 = F(vx, vy)
```

```

J = np.zeros((2, 2))

# derivative wrt vx
F_vx = F(vx + h, vy)
J[:,0] = (F_vx - F0)/h

# derivative wrt vy
F_vy = F(vx, vy + h)
J[:,1] = (F_vy - F0)/h
return J

# Iterative shooting method
def shooting_method(vx_guess, vy_guess, tol=1e-6, max_iter=20):

    vx, vy = vx_guess, vy_guess

    for iter in range(max_iter):
        F_val = F(vx, vy)
        err = np.linalg.norm(F_val)
        if err < tol:
            break
        J = shooting_jacobian(vx, vy)
        dV = -F_val @ np.linalg.inv(J)

        vx, vy = vx + dV[0], vy + dV[1]
        u_f = final_state(vx, vy)
        vx_f, vy_f = u_f[2], u_f[3]

    return vx, vy, vx_f, vy_f, err

```

```

[4]: %%time
# Initial conditions
vx_guess, vy_guess = 200, 500
vx, vy, vx_f, vy_f, err = shooting_method(vx_guess, vy_guess)

print(f'Initial velocity (vx_0, vy_0) = {vx, vy}')
print(f'Final velocity (vx_f, vy_f) = {vx_f, vy_f}')
print(f'Discrepancy |F| = {err}')

```

```

Initial velocity (vx_0, vy_0) = (128.000000000000318, 33.99999999999978)
Final velocity (vx_f, vy_f) = (128.000000000000318, -63.999999999999645)
Discrepancy |F| = 4.092727144842969e-11
CPU times: total: 172 ms
Wall time: 192 ms

```

As we can see above, we get fairly accurate results for the initial and final velocities (upto $\sim 10^{-12}$ m/s) and the discrepancy of the final position of the numerical solution is $\sim 10^{-11}$ m. The speed of the algorithm is sensitive to the step size chosen for the numerical integrator, but since

we use RK4 we can choose a fairly large step size ($dt = 0.01$ s, $N = 1000$ steps) without sacrificing precision.

1.2 Relaxation Method

In this method, we divide the total time into M divisions where every component of $\vec{u}$ (which has say, N components) will be defined at each time point, giving us a mesh of size $N \times M$. We use the same differential equation as the shooting method, this time written as a difference equation:

$$\frac{d\vec{u}}{dt} = \vec{f}(\vec{u}) \implies \frac{\vec{u}_k - \vec{u}_{k-1}}{t_k - t_{k-1}} = \vec{f}\left(\frac{t_k + t_{k-1}}{2}, \frac{\vec{u}_k + \vec{u}_{k-1}}{2}\right)$$

Hence, the correct values of $\vec{u}_k$ must satisfy the equation:

$$(\vec{u}_k - \vec{u}_{k-1}) - (t_k - t_{k-1})\vec{f}\left(\frac{t_k + t_{k-1}}{2}, \frac{\vec{u}_k + \vec{u}_{k-1}}{2}\right) = 0$$

However, we only know the partial vectors $\vec{u}_0$ and $\vec{u}_N$ (with no information about their velocities). Hence, we choose random guess values for every $\vec{u}_k$ at every time step, and define an error vector E_k :

$$\vec{E}_k(\vec{u}_k, \vec{u}_{k-1}) = (\vec{u}_k - \vec{u}_{k-1}) - (t_k - t_{k-1})\vec{f}\left(\frac{t_k + t_{k-1}}{2}, \frac{\vec{u}_k + \vec{u}_{k-1}}{2}\right)$$

For random guess values $\vec{E}_k \neq 0$, so we add correction factors such that $\vec{E}_k(\vec{u}_k + \Delta\vec{u}_k, \vec{u}_{k-1} + \Delta\vec{u}_{k-1}) = 0$. In order to determine these correction vectors, we find the first order Taylor expansion of this expression:

$$\sum_{n=1}^N \frac{\partial E_{j,k}}{\partial u_{n,k}} \Delta u_{n,k} + \sum_{n=1}^N \frac{\partial E_{j,k}}{\partial u_{n,k-1}} \Delta u_{n,k-1} = -E_{j,k} \text{ where } j = 1, 2, \dots, N, \quad k = 2, 3, \dots, M$$

Finding these derivatives analytically and solving the above as a matrix equation for each time step t_k allows us to determine the correction vectors $\Delta\vec{u}_k$ and $\Delta\vec{u}_{k-1}$. At the boundaries, we have the equations:

$$\sum_{n=1}^N \frac{\partial E_{j,1}}{\partial u_{n,1}} \Delta u_{n,1} = -E_{j,1} \text{ where } j = n_2, n_2 + 1, \dots, N$$

$$\sum_{n=1}^N \frac{\partial E_{j,M+1}}{\partial u_{n,M}} \Delta u_{n,M} = -E_{j,M+1} \text{ where } j = 1, 2, \dots, n_2$$

Where n_1 and n_2 are the number of boundary conditions defined at t_1 and t_2 , such that $n_1 + n_2 = N$. These derivatives can be used to iteratively update the guess values and converge to a lower error, thereby approximating our trajectory along the mesh points. Note that this method only minimizes the function along the mesh points instead of solving the differential equation, so the trajectory will only be defined at discrete points rather than a continuous line.

We can determine the jacobians at each time step analytically by looking at the derivative of each term of $\vec{E}_k$ with respect to each term of $\vec{u}_k$ and $\vec{u}_{k-1}$. This gives us the following matrices:

$$\frac{\partial E_{j,k}}{\partial u_{n,k}} = J_k = \begin{pmatrix} 1 & 0 & -\Delta t/2 & 0 \\ 0 & 1 & 0 & -\Delta t/2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\frac{\partial E_{j,k-1}}{\partial u_{n,k-1}} = J_{k-1} = \begin{pmatrix} -1 & 0 & -\Delta t/2 & 0 \\ 0 & -1 & 0 & -\Delta t/2 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$$

Similar reduced matrices are found at the boundaries, where the known values remain frozen. We can combine these jacobians to get a global system of equations which can be written as $J\Delta\vec{U} = -\vec{E}$, which when expanded looks like:

$$\begin{pmatrix} J_1 & J_2 & 0 & 0 & \cdots & 0 \\ 0 & J_2 & J_3 & 0 & \cdots & 0 \\ 0 & 0 & J_3 & J_4 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & J_M & J_{M+1} \end{pmatrix} \begin{pmatrix} \Delta\vec{u}_1 \\ \Delta\vec{u}_2 \\ \vdots \\ \Delta\vec{u}_M \end{pmatrix} = - \begin{pmatrix} \vec{E}_1 \\ \vec{E}_2 \\ \vdots \\ \vec{E}_{M+1} \end{pmatrix}$$

This is finally inverted and solved to determine the correction vector $\Delta\vec{U}$ which is added to the guess values to give the “relaxed” solution, which should have a smaller residual error vector $|\vec{E}|$.

```
[5]: '''
Iterative relaxation method
'''
# Parameters
g = 9.8
T = 10
M = 5
dt = T/M
N = 4

# Boundary conditions
x0, y0 = 0, 0
xM, yM = 1280, -150

# Derivative function
def f(u):
    x, y, vx, vy = u
    return np.array([vx, vy, 0, -g])

# Jacobian of f wrt u
def Jf(u):
```

```

J = np.zeros((N, N))
J[0, 2] = 1.0
J[1, 3] = 1.0
return J

# Residual vector E
def Ek(uk, ukm1, dt):
    f_mid = f(0.5*(uk+ukm1))
    return (uk-ukm1) - dt*f_mid

# Jacobian of E wrt u
def relaxation_jacobians(uk, ukm1, dt):
    Jf_mid = Jf(0.5*(uk+ukm1))
    A = np.eye(len(uk)) - dt*Jf_mid*0.5
    B = -np.eye(len(uk)) - dt*Jf_mid*0.5
    return A, B

# Assemble global system of equations
def assemble_system(U):
    U = U.reshape((M+1, N))
    E = np.zeros(M*N+4)
    J = np.zeros((M*N+4, (M+1)*N))

    for k in range(1, M+1):
        uk = U[k]
        ukm1 = U[k-1]

        E[(k-1)*N:(k-1)*N+N] = Ek(uk, ukm1, dt)

        A, B = relaxation_jacobians(uk, ukm1, dt)
        J[(k-1)*N:(k-1)*N+N, k*N:(k+1)*N] = A
        J[(k-1)*N:(k-1)*N+N, (k-1)*N:k*N] = B

    # Boundary conditions
    E[M*N+0] = U[0,0] - x0
    E[M*N+1] = U[0,1] - y0
    E[M*N+2] = U[M,0] - xM
    E[M*N+3] = U[M,1] - yM

    J[M*N+0, 0] = 1.0
    J[M*N+1, 1] = 1.0
    J[M*N+2, M*N+0] = 1.0
    J[M*N+3, M*N+1] = 1.0

    return E, J

# Iterative relaxation method

```

```

def relaxation_method(U0, tol=1e-6, max_iter=20):
    U = U0.copy()
    for iter in range(max_iter):
        E, J = assemble_system(U)
        E_norm = np.linalg.norm(E)

        if E_norm < tol:
            break

        dU = np.linalg.solve(J, -E)
        U += dU

    return U.reshape((M+1, N)), E_norm

```

```

[9]: %%time
# Initialize mesh with linear interpolation between boundary conditions
def initialize_mesh():
    U = np.zeros((M+1, N))
    for k in range(M+1):
        alpha = k/M
        U[k,0] = (1-alpha)*x0 + alpha*xM
        U[k,1] = (1-alpha)*y0 + alpha*yM
        U[k,2] = 0.0
        U[k,3] = 0.0
    return U

U0 = initialize_mesh().flatten()
U, err = relaxation_method(U0)
x0_val, y0_val, vx0, vy0 = U[0]
xM_val, yM_val, vxM, vyM = U[-1]

print("Initial velocity (vx_0, vy_0):", (vx0, vy0))
print("Final velocity (vx_M, vy_M):", (vxM, vyM))
print(f"Discrepancy |E| = {err}")

```

```

Initial velocity (vx_0, vy_0): (127.99999999999996, 34.000000000000001)
Final velocity (vx_M, vy_M): (128.00000000000003, -64.00000000000001)
Discrepancy |E| = 1.1604927132012986e-13
CPU times: total: 0 ns
Wall time: 1.01 ms

```

Clearly, the relaxation method is much faster than the shooting method if one is interested in the behaviour at only the initial and final positions. Since the intermediate trajectory is only determined up to the resolution of the mesh chosen, one can significantly speed up the algorithm by choosing a very small mesh size ($M = 5$, in this case). With this method, the velocity accuracy is comparable to the shooting method (upto $\sim 10^{-12}$ m/s) in just over 1 ms, which is a 400-fold improvement in computational time.